CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification

Hanrong Zhang[1*] **Shicheng Fan**[1*] **Henry Peng Zou**[1] **Yankai Chen**[2,3†]
Zhenting Wang[2] **Jiayu Zhou**[4] **Chengze Li**[1] **Wei-Chieh Huang**[1] **Yifei Yao**[5]
Kening Zheng[1] **Xue (Steve) Liu**[2,3] **Xiaoxiao Li**[6] **Philip S. Yu**[1]

[1]University of Illinois Chicago [2]MBZUAI [3]McGill University
[4]Columbia University [5]Zhejiang University [6]University of British Columbia
{hzhan135, psyu}@uic.edu xiaoxiao.li@ece.ubc.ca

arXiv:2604.01687v3 [cs.AI] 10 Aug 2026

Abstract

Anthropic proposes the concept of skills for LLM agents to tackle multi-step professional tasks that simple tool invocations cannot address. A tool is a single, self-contained function, whereas a skill is a structured bundle of interdependent multi-file artifacts. Currently, skill generation is not only label-intensive due to manual authoring, but also may suffer from human–machine cognitive misalignment, which can lead to degraded agent performance, as evidenced by evaluations on SkillsBench. Therefore, we aim to enable agents to autonomously generate skills. However, existing self-evolving methods designed for tools cannot be directly applied to skills due to their increased complexity. To address these issues, we propose CoEvoSkills, a self-evolving skills framework that enables agents to autonomously construct complex, multi-file skill packages. Specifically, CoEvoSkills couples a Skill Generator that iteratively refines skills with a Surrogate Verifier that co-evolves to provide informative and actionable feedback without access to ground-truth test content. On SkillsBench, CoEvoSkills outperforms five baselines on both Claude Code and Codex, and generalizes strongly to six additional LLMs. The code is publicly available at https://github.com/Zhang-Henry/CoEvoSkills.

1 Introduction

LLM agents have advanced rapidly in reasoning, planning, memory, and interaction with humans and their environments (Yao et al., 2023; Zhang et al., 2025; Luo et al., 2025; Zou et al., 2026a; Huang et al., 2026a; Wu et al., 2026; Zou et al., 2026b; Huang et al., 2026b; Min et al., 2026; 2025; Luo et al., 2026b; Li et al., 2026b; Luo et al., 2026a; Qiu et al., 2026b). Among the key drivers of this progress is the ability to invoke external tools and APIs (Schick et al., 2023; Qin et al., 2024; Patil et al., 2024). However, professional open-ended tasks, such as complex software repair, multi-step scientific analysis, and enterprise data pipeline orchestration, require far more than isolated tool invocations. Agents must orchestrate a coherent procedure across multiple steps and artifacts: decomposing goals, coordinating tools, recovering from failures, and validating intermediate outputs. This is profoundly challenging because decisions are long-horizon, instructions and scripts are tightly coupled, and reliable environmental feedback is often sparse or delayed.

To bridge the gap, Anthropic proposed the concept of agent *skills* (Anthropic, 2025c). Fig. 1 illustrates the difference between a tool and a skill: a tool is usually a simple function, whereas a skill is a structured package of workflow instructions, executable scripts, and domain references (Xu & Yan, 2026). According to the systematic evaluation presented in SkillsBench (Li et al., 2026c), equipping agents with well-crafted skills yields consistent

[*]Equal contribution.
[†]Corresponding author: yankaichen@acm.org

performance gains across a broad spectrum of professional domains, including software engineering and scientific analysis etc., confirming that structured procedural guidance substantially augments task-solving capability beyond what bare tool access affords.

Despite the demonstrated utility of skills, the prevailing paradigm relies almost entirely on human authoring, a process that is **both labor-intensive and difficult to scale, with no systematic guarantee of output quality.** As demonstrated in Fig. 6, the SkillsBench evaluation (Li et al., 2026c) reveals that human-curated skills yield highly uneven gains: while certain domains benefit substantially, others, such as Natural Science, even exhibit degraded performance after skill integration. We hypothesize that a key driver of this inconsistency is **human–machine cognitive misalignment**: workflows and abstractions designed to be intuitive for human experts do not naturally match how LLM agents process context, reason, and act under execution constraints.

To reduce manual effort, recent approaches have shifted from pre-defining static tools or APIs to self-evolve tools or tools libraries by the LLM agent itself (Chen et al., 2026; Li et al., 2026a; Lu et al., 2026; Wang et al., 2024; Xia et al., 2025). However, these methods suffer from a fundamental *tool–skill gap*: they are inherently designed for one-shot generation of simple, self-contained functions, and **are inadequate for the creation of structured, multi-file skill packages** that coordinate workflow instructions, executable scripts, and domain references across multiple artifacts. Moreover, EvoSkill provides ground-truth answers for failure diagnosis and uses held-out validation performance for skill selection (Alzubi et al., 2026), limiting applicability in real-world settings where such supervision is unavailable.

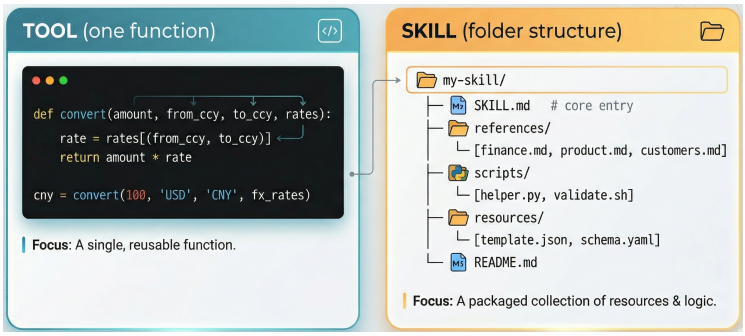


Figure 1: Tool–skill difference illustration.

To address these challenges, we propose a self-evolving skills framework *CoEvoSkills*. To overcome the inherent unreliability of one-shot multi-file skill generation, we design a main component, the **`Skill Generator`**, to iteratively generate and refine skill bundles, with skill quality steadily improving across evolution rounds, as shown in Fig. 2. It also maintains a persistent conversation context that accumulates high-fidelity feedback from another main component, the **`Surrogate Verifier`**, across iterations. The `Surrogate Verifier`, a separate LLM session without inheriting the generator's biases, is designed to address the lack of ground-truth feedback in the real world. It synthesizes tests from task instructions, observable task inputs, and generated output artifacts, providing high-fidelity feedback that drives skill co-evolution.

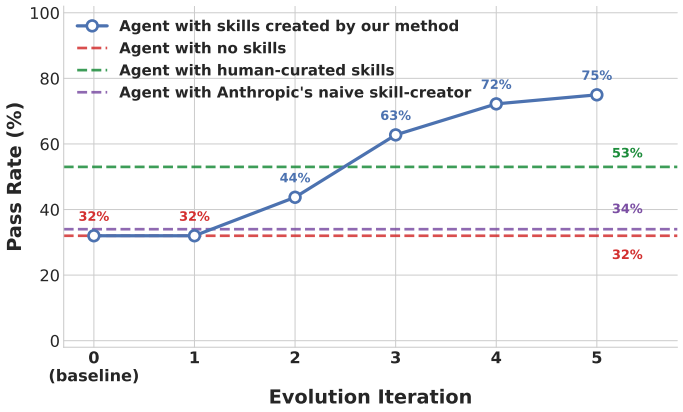


Figure 2: Skill quality improvement across 5 evolution rounds. *CoEvoSkills* surpasses human-curated skills within 5 evolution iterations.

In summary, our key contributions are as follows: ❶ We introduce *CoEvoSkills*, a co-evolutionary framework for LLM agents to self-evolve robust multi-file skill packages. ❷ We demonstrate key insights on autonomous skill generation: (i) *agents create better skills than human-curated ones* by capturing the reasoning patterns and tool-use strategies that agents actually need; (ii) *self-evolved skills are portable across different model families*, as the evolved packages encode reusable task structure rather than model-specific artifacts. ❸ We conduct extensive experiments on SkillsBench, *CoEvoSkills* achieves the highest pass rate of 71.1% (+40.5pp over the no-skill baseline), substantially surpassing all five baselines. Furthermore, skills evolved by a single frontier LLM transfer effectively to six additional LLMs from five companies, yielding gains of approximately 35–44 percentage points over their respective no-skill baselines.

2 Related Work

LLM Agent Skills. Anthropic introduced *Agent Skills* (Anthropic, 2025c) as shown in Fig. 1. A recent systematization (Jiang et al., 2026) further distinguishes skills from atomic tools and one-off plans, defining them as reusable modules with explicit applicability and termination conditions. SkillsBench (Li et al., 2026c) provides a systematic benchmark for evaluating reusable agent skills across diverse professional domains with deterministic task verifiers. Several learning-based approaches attempt to close this gap. SAGE (Wang et al., 2026) trains agents on chains of related tasks with a skill-integrated reward, yet the resulting skills remain single-file programmatic functions rather than structured multi-file packages. SkillRL (Xia et al., 2026) distills trajectories into a hierarchical skill bank via reinforcement learning, but relies on teacher-guided distillation and produces prompt-level heuristics rather than executable artifacts. *CoEvoSkills* instead generates structured, multi-file skill packages and evolves them through iterative verification.

Self-evolving LLM Agents. A growing body of work automates the self-improvement of agent capabilities, yet existing methods exhibit two recurring shortcomings. First, most self-evolving pipelines produce only single tools or function APIs or just prompt heuristics (Wang et al., 2024; Li et al., 2026a; Xia et al., 2025; Lu et al., 2026; Chen et al., 2026), and none can construct the multi-file structure a full skill package demands. Moreover, AutoSkill (Yang et al., 2026) and AutoRefine (Qiu et al., 2026a) extract reusable knowledge as prompt templates rather than executable packages, while SEAgent (Sun et al., 2025) internalizes capabilities into model weights, making them non-inspectable and non-transferable (Jiang et al., 2026). Second, some methods heavily rely on ground-truth signals for failure diagnosis, limiting applicability when such supervision is unavailable (Alzubi et al., 2026; Sun et al., 2025). *CoEvoSkills* addresses both limitations by evolving structured multi-file skill packages through isolated surrogate verification, yielding actionable diagnostics without ground-truth signals.

3 Method

3.1 Method Overview

To answer the research question posed in Sec. 1, two core difficulties must be overcome: (1) generating a multi-file skill bundle in a single pass is inherently unreliable; and (2) the agent lacks ground-truth feedback during self-evolution. *CoEvoSkills* addresses both challenges by co-evolving the agent's skill and a corresponding surrogate verifier. Fig. 3 illustrates the overall framework, and Alg. 1 formalizes the complete co-evolutionary procedure. Given a task input, the **`Skill Generator`** produces candidate skills and executes them to obtain task outputs. An informationally isolated **`Surrogate Verifier`** then independently generates and evolves test assertions against those outputs, providing structured failure diagnostics back to the generator. The two components co-evolve through **iterative generate–verify–refine** cycles: whenever the surrogate tests pass, a ground-truth oracle test re-executes the skill in a fresh environment and returns only an opaque success/failure signal. If the oracle passes, the final evolved skill is deployed to the target LLM agent (e.g.,

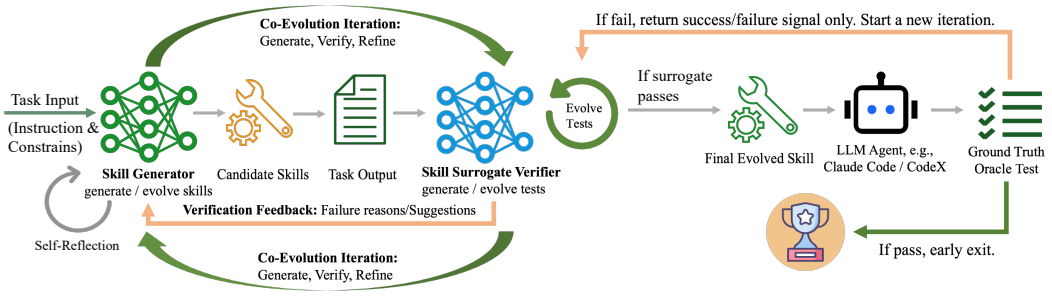


Figure 3: Overview of the *CoEvoSkills* co-evolutionary framework. The `Skill Generator` and `Surrogate Verifier` co-evolve through iterative refinement. The verifier provides structured failure feedback to drive skill improvement, while a ground-truth oracle test returns only an opaque pass/fail signal, triggering test escalation and ensuring strict information isolation.

Claude Code (Anthropic, 2025a) and Codex (OpenAI, 2025a)); otherwise, the signal triggers a new co-evolution iteration, in which the verifier escalates its tests and the generator refines the skill accordingly. Next, we formalize *CoEvoSkills* in Sec. 3.2 and Sec. 3.3.

3.2 Problem Formulation

Task Definition. Since the LLM agent never knows what the held-out ground-truth tests actually check, i.e., the success criteria remain entirely hidden, we define the task environment as a POMDP $\mathcal{M} = \langle \mathcal{X}, \mathcal{A}, \mathcal{T}, \mathcal{O}, \Omega, \mathcal{R} \rangle$. Here, $\mathcal{X}$ is the underlying state space, i.e., the complete filesystem and processes, $\mathcal{A}$ comprises the agent's actions, i.e., terminal commands and file edits, $\mathcal{T}(x' \mid x, a)$ is the deterministic state transition upon executing action a in state x and reaching successor state x', $\mathcal{O}$ is the observation space, i.e., command execution results, $\Omega(o \mid x, a)$ maps post-action states to partial observations, and y_H denotes the output artifacts contained in terminal state x_H, while $\mathcal{R}(y_H) \in [0, 1]$ evaluates those artifacts against hidden ground-truth tests. Because the agent only receives partial observations $o_t \sim \Omega(\cdot \mid x_t, a_t)$, it acts based on the observation–action history $h_t = (o_1, a_1, \ldots, a_{t-1}, o_t)$.

Optimization Objective. Unlike atomic tools that offer simple functional interfaces, a *skill* $\mathcal{S}$ (Anthropic, 2025c) is a structured bundle of domain-specific instructions, executable scripts, and reference materials that collectively guide an agent in navigating a task space. The skill conditions the agent's policy:

$$a_t \sim \pi_\theta(a_t \mid h_t, \mathcal{S}), \tag{1}$$

where π_θ is an LLM policy. We define the expected terminal reward under skill $\mathcal{S}$ as:

$$J(\mathcal{S}) \triangleq \mathbb{E}_{\tau \sim P(\tau \mid \pi_\theta, \mathcal{S}, \mathcal{M})}[\mathcal{R}(y_H)], \tag{2}$$

where H is the terminal horizon and $\tau = (o_1, a_1, \ldots, a_{H-1}, o_H)$ is the execution trajectory. Our objective is to discover an optimal skill $\mathcal{S}^*$ that maximizes J:

$$\mathcal{S}^* = \arg\max_{\mathcal{S}} J(\mathcal{S}). \tag{3}$$

3.3 *CoEvoSkills* Framework

However, directly optimizing $J(\mathcal{S})$ (Eq. 3) is intractable: ground-truth evaluation is computationally expensive and returns only an opaque pass/fail signal; no test content or failure details are revealed to the agent. To provide dense, actionable feedback, we introduce a surrogate verifier reward $\tilde{\mathcal{R}}(y, \mathcal{V})$, defined by a suite of deterministic test assertions $\mathcal{V} = \{e_1, \ldots, e_{|\mathcal{V}|}\}$ generated by an independent verifier

$$\tilde{\mathcal{R}}(y, \mathcal{V}) \triangleq \frac{1}{|\mathcal{V}|} \sum_{k=1}^{|\mathcal{V}|} \mathbf{1}[e_k(y)] \in [0, 1], \tag{4}$$

Algorithm 1 *CoEvoSkills* co-evolution Algorithm

Require: Instruction I, observable task inputs $\mathcal{D}$, background knowledge $\mathcal{B}$, environment $\mathcal{E}$, meta-skill $\mathcal{S}_{\text{meta}}$ (`skill-creator`)
Require: Ground-truth oracle rounds K=5, surrogate repair retries M=15, context cap β=0.7
Require: LLM policy π_θ (generator), independent verifier policy π_θ^V
Ensure: Final evolved skill $\mathcal{S}^*$

$C \leftarrow (I, \mathcal{B}, \mathcal{S}_{\text{meta}})$ // in: instruction I, background knowledge $\mathcal{B}$, meta-skill; out: initial context C
$\mathcal{S}^{(0)} \sim \pi_\theta(\cdot \mid C)$ // in: context C; out: skill bundle $\mathcal{S}^{(0)}$ (code + SKILL.md)
$\mathcal{V}^{(0)} \leftarrow \perp$ // uninitialized surrogate verifier test suite
$i \leftarrow 0;\ j \leftarrow 0;\ n \leftarrow 0;\ r \leftarrow 0$ // skill ver.; test ver.; oracle rounds; repair retries
$\mathcal{R}_{\text{best}} \leftarrow 0;\ \mathcal{S}^* \leftarrow \mathcal{S}^{(0)}$ // $\mathcal{R}_{\text{best}}$: best oracle score so far
while $n < K$ **and** $r < M$ **do**
  // — `Skill Generator`: *execute and produce outputs* —
  $y^{(i)} \leftarrow \Phi(\mathcal{S}^{(i)}, \mathcal{E})$ // Φ: execute skill in env and collect outputs; out: artifacts $y^{(i)}$
  if LLM context usage proportion $> \beta$ **then**
    break // prevent LLM context overflow
  end if
  if $j = 0$ **and** $\mathcal{V}^{(0)} = \perp$ **then**
    $\mathcal{V}^{(0)} \sim \pi_\theta^V\left(\cdot \mid I, \mathcal{D}, y^{(i)}\right)$ // initialize tests from instruction, observable inputs, and artifacts
  end if
  // — `Surrogate Verifier`: *evaluate and refine (Eq. 4, Eq. 5)* —
  $\tilde{\mathcal{R}}^{(i,j)} \leftarrow \tilde{\mathcal{R}}(y^{(i)}, \mathcal{V}^{(j)})$ // $\tilde{\mathcal{R}}$: surrogate reward; in: artifacts, tests; out: pass rate $\in [0,1]$
  if $\tilde{\mathcal{R}}^{(i,j)} < 1$ **then**
    $\mathcal{F}^{(i,j)} \sim \pi_\theta^V\left(\cdot \mid I, \mathcal{D}, y^{(i)}, \mathcal{V}^{(j)}\right)$ // in: I, inputs $\mathcal{D}$, artifacts $y^{(i)}$, tests $\mathcal{V}^{(j)}$; out: diagnostic $\mathcal{F}$
    $C \leftarrow C \oplus \mathcal{F}^{(i,j)}$ // append error diagnostic $\mathcal{F}$ to generator context C
    $\mathcal{S}^{(i+1)} \sim \pi_\theta\left(\cdot \mid \mathcal{S}^{(i)}, C\right)$ // skill refinement (Eq. 7)
    $i \leftarrow i{+}1;\ r \leftarrow r{+}1;$ **continue** // evolve $\mathcal{S}$; $\mathcal{V}^{(j)}$ locked
  end if
  // — *Ground-Truth Oracle Test: independent re-execution in fresh environment* —
  $\hat{y}^{(i)} \leftarrow \Phi(\mathcal{S}^{(i)}, \mathcal{E}')$ // in: skill $\mathcal{S}^{(i)}$, fresh env $\mathcal{E}'$; out: artifacts $\hat{y}^{(i)}$
  $\mathcal{R}^{(i)} \leftarrow \mathcal{R}(\hat{y}^{(i)});\ n \leftarrow n{+}1$ // $\mathcal{R}$: ground-truth oracle reward; out: score $\in [0,1]$
  if $\mathcal{R}^{(i)} = 1$ **then**
    $\mathcal{S}^* \leftarrow \mathcal{S}^{(i)};$ **return** $\mathcal{S}^*$ // early exit: perfect score
  else if $\mathcal{R}^{(i)} > \mathcal{R}_{\text{best}}$ **then**
    $\mathcal{R}_{\text{best}} \leftarrow \mathcal{R}^{(i)};\ \mathcal{S}^* \leftarrow \mathcal{S}^{(i)}$ // save best snapshot
  end if
  // — *Co-evolution: test escalation (Eq. 6, Eq. 8)* —
  $C \leftarrow C \oplus \mathbf{1}[\mathcal{R}^{(i)}{<}1]$ // $\mathbf{1}[\cdot]$: indicator fn; append oracle pass/fail bit to C (no test content)
  $\mathcal{V}^{(j+1)} \sim \pi_\theta^V\left(\cdot \mid I, \mathcal{D}, y^{(i)}, \mathcal{V}^{(j)}\right)$ // verifier escalation (Eq. 8)
  $j \leftarrow j{+}1$ // $\mathcal{V}$ evolves; $\mathcal{S}^{(i)}$ re-evaluated next iteration
end while
return $\mathcal{S}^*$ // save best skill

where y denotes the output artifacts produced by skill execution and $\mathbf{1}[e_k(y)]$ indicates whether assertion e_k passes on them. However, the surrogate is only useful insofar as it faithfully approximates the hidden $\mathcal{R}$. This creates a coupled optimization problem: the skill must maximize a proxy that itself must be aligned with the hidden ground truth. Since the ground-truth (GT) oracle test reveals only a binary pass/fail signal $\mathbf{1}[\mathcal{R}(\hat{y}^{(i)}) < 1]$, where $\hat{y}^{(i)}$ denotes the oracle's independent re-execution output, and no test content, we cannot directly optimize against $\mathcal{R}$. Instead, letting I denote the task instruction, we define the rollout operator $\Phi(\mathcal{S}, \mathcal{E})$ as the execution output obtained by rolling out $\pi_\theta(\cdot \mid h_t, \mathcal{S})$ in environment $\mathcal{E}$, so that $y^{(i)} = \Phi(\mathcal{S}^{(i)}, \mathcal{E})$ is the output of the i-th skill version and $\hat{y}^{(i)} = \Phi(\mathcal{S}^{(i)}, \mathcal{E}')$ is the output produced by an independent oracle re-execution in a fresh environment $\mathcal{E}'$. $\mathcal{V}^{(j)}$ is the j-th version of the surrogate verifier test suite. We use $\mathcal{D}$ for all

inputs available during skill evolution, including instance data and background documents $\mathcal{B}$ ($\mathcal{B} \subseteq \mathcal{D}$); $\mathcal{D}$ contains no hidden oracle information. *CoEvoSkills* proceeds via alternating refinement:

$$\textit{Skill refinement:} \quad \mathcal{S}^{(i+1)} \leftarrow \arg\max_{\mathcal{S}} \tilde{\mathcal{R}}\big(\Phi(\mathcal{S}, \mathcal{E}),\ \mathcal{V}^{(j)}\big), \tag{5}$$

$$\textit{Test escalation:} \quad \mathcal{V}^{(j+1)} \sim \pi_\theta^V\big(\cdot \mid I,\ \mathcal{D},\ y^{(i)},\ \mathcal{V}^{(j)}\big), \ \text{ if } \mathbf{1}\Big[\tilde{\mathcal{R}}(y^{(i)}, \mathcal{V}^{(j)})=1 \ \wedge\ \mathcal{R}(\hat{y}^{(i)})<1\Big] \tag{6}$$

In practice, the arg max in Eq. 5 is approximated by iterative LLM sampling (Eq. 7). Skill refinement maximizes $\tilde{\mathcal{R}}$ under a fixed verifier test suite $\mathcal{V}^{(j)}$; test escalation is triggered only when the oracle's binary signal exposes a gap between $\tilde{\mathcal{R}}$ and $\mathcal{R}$, forcing the verifier to independently strengthen its tests without any access to ground-truth test content. *CoEvoSkills* realizes this alternating optimization through three informationally isolated components: a `Skill Generator` and a `Surrogate Verifier` orchestrated in a co-evolutionary loop (Alg. 1).

`Skill Generator`. A single-pass skill generation often produces bundles with coverage gaps and logical errors, because the agent lacks ground-truth feedback during generation. To enable iterative refinement, the `Skill Generator` maintains a persistent conversation context C. We study *agent-driven skill evolution*, in which the generator begins with task-relevant background knowledge available at initialization, such as organizational documentation or domain references. Accordingly, $C^{(0)} = (I, \mathcal{B}, \mathcal{S}_{\text{meta}})$, where I is the task instruction, $\mathcal{B}$ denotes this task-relevant background knowledge, and $\mathcal{S}_{\text{meta}}$ is a domain-agnostic meta-skill (`skill-creator`) that teaches how to create skills. At each revision, the LLM π_θ (Eq. 1) updates the current skill $\mathcal{S}^{(i)}$ using accumulated surrogate-verifier diagnostics:

$$\mathcal{S}^{(i+1)} \sim \pi_\theta\Big(\cdot \mid \mathcal{S}^{(i)},\ C^{(i+1)}\Big), \quad C^{(i+1)} = C^{(i)} \oplus \mathcal{F}^{(i,j)}, \tag{7}$$

where $\mathcal{S}^{(i)}$ is the i-th skill version, $\mathcal{F}^{(i,j)}$ is the failure diagnostic from the `Surrogate Verifier` after evaluating $\mathcal{S}^{(i)}$ under test suite $\mathcal{V}^{(j)}$, comprising failed test cases, root-cause analysis, and actionable revision suggestions, and $\oplus$ appends detailed feedback to the LLM context. The agent executes $\mathcal{S}^{(i)}$ via rollout $y^{(i)} = \Phi(\mathcal{S}^{(i)}, \mathcal{E})$ (line 8), and the resulting outputs are passed to the `Surrogate Verifier` for evaluation.

`Surrogate Verifier`. Since the ground-truth reward $\mathcal{R}$ returns only an opaque pass/fail signal, the `Skill Generator` lacks dense feedback to diagnose and correct errors. The `Surrogate Verifier` addresses this gap by serving as a proxy for $\mathcal{R}$, providing per-assertion failure diagnostics that the opaque oracle cannot. To reduce confirmation bias, the `Surrogate Verifier` uses an independent LLM session π_θ^V and receives only I, $\mathcal{D}$ (including $\mathcal{B}$), $y^{(i)}$, and $\mathcal{V}^{(j)}$. It has no access to generator reasoning or skill source, or to hidden oracle criteria, test content, or diagnostics, while remaining able to construct input-grounded tests. The verifier generates a proxy verifier test suite $\mathcal{V} = \{e_1, \ldots, e_{|\mathcal{V}|}\}$ of deterministic assertions, yielding the proxy reward $\tilde{\mathcal{R}}(y, \mathcal{V})$ defined in Eq. 4. After the first rollout, it initializes $\mathcal{V}^{(0)}$ from $(I, \mathcal{D}, y^{(0)})$ before computing the first surrogate reward (Alg. 1, lines 12–16). It then iteratively refines this verifier test suite by reading the previous script $\mathcal{V}^{(j)}$ and current outputs $y^{(i)}$:

$$\mathcal{V}^{(j+1)} \sim \pi_\theta^V\big(\cdot \mid I,\ \mathcal{D},\ y^{(i)},\ \mathcal{V}^{(j)}\big), \tag{8}$$

where the verifier conditions on the task instruction I, observable task inputs $\mathcal{D}$, the agent's current outputs $y^{(i)}$, and its own previous test script $\mathcal{V}^{(j)}$ to produce an improved verifier test suite $\mathcal{V}^{(j+1)}$. When the surrogate reward indicates failure ($\tilde{\mathcal{R}} < 1$), the verifier additionally generates a structured failure diagnostic $\mathcal{F}^{(i,j)} \sim \pi_\theta^V(\cdot \mid I, \mathcal{D}, y^{(i)}, \mathcal{V}^{(j)})$, including per-assertion results, root-cause analysis, and actionable revision suggestions fed back to the `Skill Generator` (Eq. 7).

Co-evolution of `Skill Generator` and `Surrogate Verifier`. The alternating optimization in Eq. 5–Eq. 6 couples two feedback pathways (Alg. 1). When the surrogate reward indicates failure ($\tilde{\mathcal{R}} < 1$), the verifier test suite $\mathcal{V}$ is held fixed and the failure diagnostic $\mathcal{F}$ drives skill revision (Eq. 7). When the surrogate test passes but the ground-truth oracle fails, only an opaque pass/fail bit is returned, i.e., no test content or failure details to prevent the `Skill Generator` from overfitting to the held-out tests. The `Surrogate Verifier` must then independently escalate its test suite (Eq. 8) according to the observable task inputs and current output artifacts. For example, it may generate more diverse, comprehensive and challenging test cases. Through this dual-feedback mechanism, the skill $\mathcal{S}$ improves under surrogate test pressure. Fig. 2 confirms convergence within few co-evolutionary iterations.

4 Experiments

In this section, we answer three Research Questions (RQs): (1) Are self-evolved skills useful, and can they outperform human-curated skills (Sec. 4.2)? (2) Can evolved skills transfer across models and agent harnesses (Sec. 4.3)? (3) How are performance gains distributed across professional domains (Sec. 4.4)? Component ablations are reported in Appendix A.

4.1 Experimental Setup

We evaluate *CoEvoSkills* on SkillsBench (Li et al., 2026c). Our experiments cover 85 tasks, providing broad coverage of the real-world distribution of skill-augmented tasks. Each task is paired with a deterministic verifier, enabling reproducible, binary pass/fail evaluation without subjective human judgment. We adopt SkillsBench as the sole evaluation suite because to the best of our knowledge, it is the only benchmark purpose-built for assessing the utility of agent *skills*. The primary metric is **pass rate**: the proportion of tasks with reward $= 1.0$, i.e., all tests passed, otherwise reward $= 0.0$. Unless otherwise stated, all conditions use the same instruction format. For conditions with pre-installed skills, the task instruction notes their availability.

We compare five baselines against *CoEvoSkills*. The **No-Skill Baseline** evaluates the agent's performance when no skills are available. **Self-Generated Skills** replicates the one-pass self-generation condition from SkillsBench (Li et al., 2026c): the agent generates one to five skill documents in a single pass before solving the task, with no iterative evolution or verification (see prompt in Sec. D.4). **CoT-Guided Self-Generation** extends the Self-Generated Skills condition with a structured five-step chain-of-thought prompt (see prompt in Sec. D.5). **Skill-Creator** adopts Anthropic's official `skill-creator` (Anthropic, 2025c). Since our evaluation is fully autonomous, we replace its human-interactive steps with autonomous equivalents (see Sec. D.3): a first session iteratively drafts, self-tests, and refines skills for at least three iterations, and a second session solves the task using the resulting skills. **Human Curated Skills** pre-installs the human-authored skill packages released with SkillsBench. For the evolution agent, we use Claude Opus 4.6 and GPT-5.2 as backbone models. Each evolution run starts with general background knowledge $\mathcal{B}$ available at initialization, without task-instance solution procedures or hidden oracle information. For baseline comparisons, each primary method is evaluated over 5 independent runs, and mean $\pm$ standard deviation is reported.

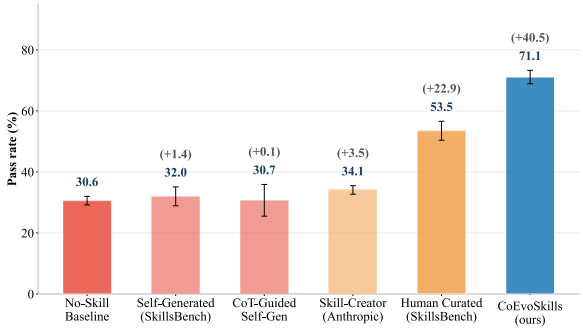


Figure 4: Skill quality comparisons with baselines on SkillsBench (Claude Opus 4.6 + Claude-Code). Error bars: ± 1 std over 5 runs.

In addition, we also evaluate the transferability of the skills evolved by Claude Opus 4.6 on six additional models: GPT-5.2 (OpenAI, 2025b), Claude Sonnet 4.5 (Anthropic, 2025d), Claude Haiku 4.5 (Anthropic, 2025b), Qwen3-Coder-480B (Yang et al., 2025), DeepSeek V3-671B (DeepSeek-AI et al., 2024) and Mistral Large 3-675B (Mistral AI, 2025). Each model is evaluated over 3 independent runs.

Component	Setting
Backbones	Claude Opus 4.6, GPT-5.2
Surrogate verifier model	Same as evolution backbone (Claude Opus 4.6 / GPT-5.2)
Ground-truth oracle test agent	`Claude-Code` (Claude Opus 4.6) / `Codex` (GPT-5.2)
Evolution stage	K=5 Ground Truth Oracle rounds, M=15 surrogate repair retries
Evolution runtime	timeout multiplier 5× (effective 3000s/task), 4 parallel workers
Evaluation stage	timeout 7200s/task, 10 parallel workers

Table 1: Shared configuration for evolution and evaluation.

Model	Agent harness
Claude Opus 4.6	`Claude-Code` (Anthropic, 2025a)
GPT-5.2	`Codex` (OpenAI, 2025a)
Claude Sonnet 4.5	`Claude-Code` (Anthropic, 2025a)
Claude Haiku 4.5	`Terminus-2` (Merrill et al., 2026)
Qwen3 Coder	`Terminus-2` (Merrill et al., 2026)
DeepSeek V3	`Terminus-2` (Merrill et al., 2026)
Mistral Large 3	`Terminus-2` (Merrill et al., 2026)

Table 2: Agent harness used for each model in ground-truth oracle evaluation.

4.2 RQ1: Skill Quality Comparison

Fig. 4 presents the core comparison on Claude Opus 4.6 with Claude-Code. *CoEvoSkills* reaches a **71.1%** pass rate, exceeding the no-skill baseline (30.6%) by +40.5pp and human-curated skills (53.5%) by +17.6pp. The Skill-Creator baseline achieves only 34.1%, barely above the no-skill baseline. Two in-session self-generation baselines perform no better: the SkillsBench self-generated skills baseline (Li et al., 2026c) reaches 32.0% (±3.1), and a CoT-guided variant that follows a structured five-step chain-of-thought prompt reaches 30.7% (±5.2), both providing negligible improvement over the no-skill baseline. This confirms that skill generation without co-evolutionary verification is insufficient regardless of the generation strategy: the gains of *CoEvoSkills* originate from the iterative verification loop, not from the skill-creation prompt itself. We summarize the main insight as follows.

Takeaway 1: Agents can create better skills than humans

Self-evolved skills outperform human-curated ones by encoding agent-native reasoning patterns, tool-use preferences, and decomposition strategies rather than following human assumptions about how agents should operate.

4.3 RQ2: Skill Cross-Model Transferability

Having established that *CoEvoSkills* produces high-quality skills on two primary LLMs, we next examine whether these skills generalize across LLM model families. As shown in Fig. 5, self-evolved skills yield substantial gains on both primary backbones: Claude Opus 4.6 (+40.5pp) and GPT-5.2 (+40.2pp). Transferring Opus-evolved skills to six additional models yields gains of approximately 35–44 percentage points across models, demonstrating that the distilled workflows are not tied to the originating model. Exact pass rates are reported in Tab. 3.

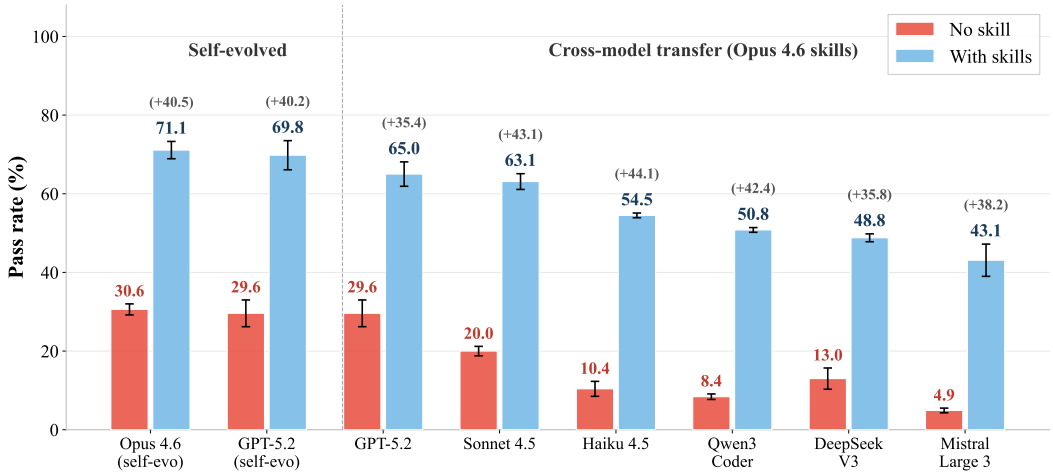


Figure 5: Cross-model skill transferability on SkillsBench. Skills evolved by Claude Opus 4.6 are transferred to six additional models spanning five providers. Each pair of bars shows the no-skill baseline (red) and the with-skills pass rate (blue). Delta annotations indicate absolute improvement. All models benefit substantially, with gains of approximately 35–44 percentage points across models, confirming that the evolved skills encode reusable task structure rather than model-specific artifacts.

Model	With skills	No skill	Δ
Self-Evolved Skills			
Claude Opus 4.6 (self-evolved)	71.1	30.6	+40.5
GPT-5.2 (self-evolved)	69.8	29.6	+40.2
Cross-Model Transfer (Opus 4.6 Evolved Skills)			
GPT-5.2	65.0	29.6	+35.4
Claude Sonnet 4.5	63.1	20.0	+43.1
Claude Haiku 4.5	54.5	10.4	+44.1
Qwen3 Coder	50.8	8.4	+42.4
DeepSeek V3	48.8	13.0	+35.8
Mistral Large 3	43.1	4.9	+38.2

Table 3: Cross-model skill transferability, pass rate (%).

Interestingly, GPT-5.2 benefits from Opus-transferred skills (65.0%), yet its own self-evolved skills (69.8%) still outperform the transferred set by 4.8pp, indicating a modest but consistent advantage for model-matched evolution. We summarize the main takeaway as follows.

Takeaway 2: Skills are portable across model families

Self-evolved skills encode reusable task structures rather than model-specific artifacts, enabling broad cross-model transferability even without model-matched evolution.

4.4 RQ3: Domain-Level Analysis

Beyond aggregate metrics, Fig. 6 reveals how gains are distributed across domains. Self-evolved skills outperform human-curated skills in 9 of 11 domains, with the largest margins in Finance (+56.9pp over human-curated) and Cybersecurity (+23.2pp). The pattern is non-uniform: domains where human-curated skills already perform well (Energy, Robotics) see diminishing returns from evolution, whereas domains where human curation provides little benefit see the largest gains. We summarize the main takeaway as follows.

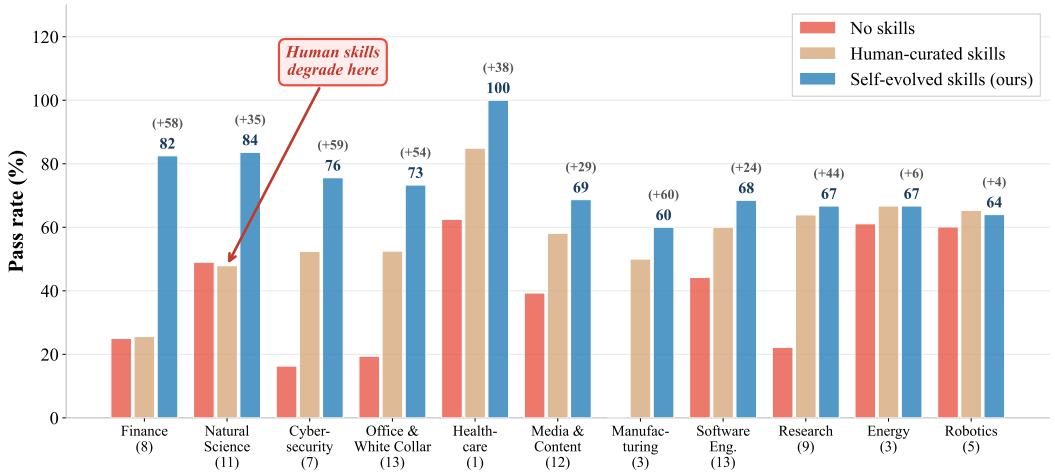


Figure 6: Per-domain pass rates on SkillsBench. Three conditions are compared using Claude Opus 4.6: no-skill baseline, human-curated skills, and *CoEvoSkills* self-evolved skills, across 11 professional domains. Numbers in parentheses indicate task counts. Self-evolved skills outperform human-curated skills in 9 of 11 domains. The arrow highlights Natural Science, where human-curated skills degrade performance, whereas self-evolved skills yield substantial gains, evidencing human–machine cognitive misalignment.

Takeaway 3: Human–machine misalignment

Human-curated skills can actively degrade agent performance in certain domains; co-evolutionary optimization bridges this gap by discovering procedures aligned with how LLM agents actually reason.

4.5 Evolution Dynamics

Fig. 2 traces the pass rate across evolution rounds against three static baselines: the no-skill baseline (30.6%), Anthropic's naive skill-creator (34.1%), and human-curated skills (53.5%). At round 0 (one-shot generation without verification), *CoEvoSkills* performs on par with the no-skill baseline, but the pass rate climbs sharply once iterative verification begins, reaching 44% at round 2, surpassing human-curated skills at round 3 (63%), and converging at 75% by round 5. This confirms that the co-evolutionary loop, not the generation prompt, is the primary driver of skill quality, and that convergence within five rounds keeps the evolution cost practical. Appendix B provides the full distribution of verification cycles and oracle rounds across all tasks: each task requires 4.1 verification cycles and 2.4 Ground Truth Oracle rounds on average to achieve convergence. Moreover, Appendix C presents a detailed trace of a representative evolution trajectory.

5 Conclusion

We presented *CoEvoSkills*, a co-evolutionary framework for agent skill self-generation. This design overcomes both the unreliability of one-shot skill generation and the lack of ground-truth feedback in real-world settings. On SkillsBench, *CoEvoSkills* substantially outperforms human-curated skills and all self-generation baselines, while demonstrating strong transferability. Our experiments reveal a human–machine cognitive misalignment that co-evolutionary optimization can bridge. However, *CoEvoSkills* evolves skills from task-relevant background knowledge available at initialization. A natural extension is open-world skill evolution, where agents autonomously seek and verify additional knowledge beyond the initial task context. For example, OpenSkill studies this broader setting by bootstrapping transferable skills and verification signals from open-world resources without target-task supervision (Yan et al., 2026). Extending *CoEvoSkills* toward this regime is a promising next step.

References

Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026.

Anthropic. Claude Code Overview. https://code.claude.com/docs/en/overview, 2025a. Accessed: 2026-08-08.

Anthropic. Claude haiku 4.5 system card. https://www.anthropic.com/claude-haiku-4-5-system-card, 2025b. Accessed: 2026-08-08.

Anthropic. Agent Skills. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview, 2025c. Accessed: 2026-08-08.

Anthropic. Claude sonnet 4.5 system card. https://www.anthropic.com/claude-sonnet-4-5-system-card, 2025d. Accessed: 2026-08-08.

Shiqi Chen, Jingze Gai, Ruochen Zhou, Jinghan Zhang, Tongyao Zhu, Junlong Li, Kangrui Wang, Zihan Wang, Zhengyu Chen, Klara Kaleb, et al. Skillcraft: Can llm agents learn to use tools skillfully? *arXiv preprint arXiv:2603.00718*, 2026.

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437*, 2024. doi: 10.48550/arXiv.2412.19437. URL https://arxiv.org/abs/2412.19437.

Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, Shanglin Wu, Ruiyao Xu, Liangwei Yang, Rui Yang, Wooseong Yang, Chin-Yuan Yeh, Hanrong Zhang, Haozhen Zhang, Siqi Zhu, Henry Peng Zou, et al. A Survey of Agent Memory in the Second Half: Towards Self-Evolving and Long-Horizon Agents, 2026a. URL https://arxiv.org/abs/2602.06052.

Wei-Chieh Huang, Henry Peng Zou, Yaozu Wu, Dongyuan Li, Yankai Chen, Weizhi Zhang, Yangning Li, Angelo Zangari, Jizhou Guo, Chunyu Miao, Liancheng Fang, Langzhou He, Yinghui Li, Renhe Jiang, and Philip S. Yu. Deep Research with Open-Domain Evaluation and Multi-Stage Guardrails for Safety. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 43403–43446, San Diego, California, United States, July 2026b. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.2010. URL https://aclanthology.org/2026.acl-long.2010/.

Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. Sok: Agentic skills–beyond tool use in llm agents. *arXiv preprint arXiv:2602.20867*, 2026.

Haotian Li, Shijun Yang, Weizhen Qi, Silei Zhao, Rui Hua, Mingzhu Song, Xiaojian Yang, and Chao Peng. Yunjue agent tech report: A fully reproducible, zero-start in-situ self-evolving agent system for open-ended tasks. *arXiv preprint arXiv:2601.18226*, 2026a.

Jialin Li, Zhenhao Chen, Hanjun Luo, and Hanan Salam. PrefIx: Understand and Adapt to User Preference in Human-Agent Interaction. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 30110–30149, San Diego, California, United States, July 2026b. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.1506. URL https://aclanthology.org/2026.findings-acl.1506/.

Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, et al. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks, 2026c. URL https://arxiv.org/abs/2602.12670.

Jiaxuan Lu, Ziyu Kong, Yemin Wang, Rong Fu, Haiyuan Wan, Cheng Yang, Wenjie Lou, Haoran Sun, Lilong Wang, Yankai Jiang, et al. Beyond static tools: Test-time tool evolution for scientific reasoning. *arXiv preprint arXiv:2601.07641*, 2026.

Hanjun Luo, Shenyu Dai, Chiming Ni, Xinfeng Li, Guibin Zhang, Kun Wang, Tongliang Liu, and Hanan Salam. AgentAuditor: Human-level Safety and Security Evaluation for LLM Agents. In *Advances in Neural Information Processing Systems*, volume 38, pp. 43241–43298. Curran Associates, Inc., 2025. doi: 10.52202/085713-1440. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3dc85735f6e2fcf093e67b134fa00d21-Abstract-Conference.html.

Hanjun Luo, Zhimu Huang, Sylvia Chung, Yiran Wang, Yingbin Jin, Jialin Li, Jiang Li, Xinfeng Li, and Hanan Salam. AtelierEval: Agentic Evaluation of Humans & LLMs as Text-to-Image Prompters, 2026a. URL https://arxiv.org/abs/2605.22645.

Hanjun Luo, Chiming Ni, Jiaheng Wen, Zhimu Huang, Yiran Wang, Bingduo Liao, Sylvia Chung, Yingbin Jin, Xinfeng Li, Wenyuan Xu, XiaoFeng Wang, and Hanan Salam. CentaurEval: Benchmarking Human-in-the-Loop Value in Agentic Coding, 2026b. URL https://arxiv.org/abs/2512.04111.

Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=a7Qa4CcHak.

Dehai Min, Zhiyang Xu, Guilin Qi, Lifu Huang, and Chenyu You. UniHGKR: Unified Instruction-aware Heterogeneous Knowledge Retrievers. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 4577–4594, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.naacl-long.234. URL https://aclanthology.org/2025.naacl-long.234/.

Dehai Min, Kailin Zhang, Tongtong Wu, and Lu Cheng. QuCo-RAG: Quantifying Uncertainty from the Pre-training Corpus for Dynamic Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 16482–16500, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.812. URL https://aclanthology.org/2026.findings-acl.812/.

Mistral AI. Introducing mistral 3. https://mistral.ai/news/mistral-3/, 2025. Accessed: 2026-08-08.

OpenAI. Introducing Codex. https://openai.com/index/introducing-codex/, 2025a. Accessed: 2026-08-08.

OpenAI. Introducing GPT-5.2. https://openai.com/index/introducing-gpt-5-2/, 2025b. Accessed: 2026-08-08.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*, volume 37, pp. 126544–126565. Neural Information Processing Systems Foundation, Inc., 2024. doi: 10.52202/079017-4020. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=dHng2O0Jjr.

Libin Qiu, Zhirong Gao, Junfu Chen, Yuhang Ye, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Shuo Tang. Autorefine: From trajectories to reusable expertise for continual llm agent refinement. *arXiv preprint arXiv:2601.22758*, 2026a.

Shikai Qiu, Xiaowen Xu, Benlei Cui, Ting Ma, Xiufeng Huang, Wenjing Jiang, Shaoxuan He, Haolei Xu, Chunyang Chai, Yujian Li, Yiliang Zhang, Guanghui Wang, Ziheng Wang,

Ziwen Xu, Zhaoyu Fan, Jinhao Chen, Ruijie Jian, Hongxing Li, Chuxi Xiao, Xinyue Chen, Wenxuan Liu, Libin Dong, Yupeng Cao, Xiaoqian Xia, Jing Wang, Zhe Jiang, Zhenan Ye, Guang Yang, Bin Liu, Wei Peng, Ziqiang Zhu, Meihui Lian, Kaiwen Lv Kacuila, Haidong Ding, Dongjie Zhang, Yangfan Zhou, Bingyu Zhu, Yan Wang, Hai Zhao, Xuan Jin, Wei Zhao, Pengfei Sun, Huiming Zhang, Wei Wang, Xipeng Cao, Jialun Chen, Xiao Chen, Shaola Ren, Yunqing Hu, Bin Li, Chengwen Yao, Meng Huang, Xianfeng Li, Bin Tang, Chao Liu, Hui Xue, Longtao Huang, and Haiwen Hong. Yuvion VL: A Multimodal Foundation Model for Adversarial Content and AI Safety, 2026b. URL https://arxiv.org/abs/2606.25034.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36: 68539–68551, 2023.

Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. Seagent: Self-evolving computer use agent with autonomous learning from experience. *arXiv preprint arXiv:2508.04700*, 2025.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=ehfRiF0R3a.

Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement Learning for Self-Improving Agent with Skill Library. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1529–1550, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.69. URL https://aclanthology.org/2026.acl-long.69/.

Zhaofen Wu, Hanrong Zhang, Fulin Lin, Wujiang Xu, Xinran Xu, Yankai Chen, Henry Peng Zou, Shaowen Chen, Weizhi Zhang, Xue Liu, Philip S. Yu, and Hongwei Wang. GAM: Hierarchical Graph-based Agentic Memory for LLM Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 34647–34664, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.1600. URL https://aclanthology.org/2026.acl-long.1600/.

Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-swe-agent: Can software engineering agents self-evolve on the fly? *arXiv preprint arXiv:2511.13646*, 2025.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.

Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.

Zhiling Yan, Dingjie Song, Hanrong Zhang, Wei Liang, Yuxuan Zhang, Yutong Dai, Lifang He, Philip S. Yu, Ran Xu, Xiang Li, and Lichao Sun. OpenSkill: Open-World Self-Evolution for LLM Agents, 2026. URL https://arxiv.org/abs/2606.06741.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*, 2026.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=V4y0CpX4hK.

Henry Peng Zou, Wei-Chieh Huang, Yaozu Wu, Jizhou Guo, Yankai Chen, Chunyu Miao, Hoang H Nguyen, Yue Zhou, Weizhi Zhang, Liancheng Fang, Hanrong Zhang, Fangxin Wang, Pengfei Zhang, Langzhou He, Yangning Li, Dongyuan Li, Renhe Jiang, and Philip S. Yu. LLM-Based Human-Agent Collaboration and Interaction Systems: A Survey. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 36335–36364, San Diego, California, United States, July 2026a. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.1811. URL https://aclanthology.org/2026.findings-acl.1811/.

Henry Peng Zou, Chunyu Miao, Wei-Chieh Huang, Yankai Chen, Yue Zhou, Hanrong Zhang, Yaozu Wu, Liancheng Fang, Zhengyao Gu, Zhen Zhang, Kening Zheng, Fangxin Wang, Yi Nian, Shanghao Li, Wenzhe Fan, Langzhou He, Weizhi Zhang, Xue Liu, and Philip S. Yu. When users change their mind: Evaluating interruptible agents in long-horizon web navigation. *arXiv preprint arXiv:2604.00892*, 2026b.

A Ablation Studies

Tab. A1 summarizes the ablation results. We examine the contribution of the surrogate verifier, background context, and the evolution process. All ablation experiments use Claude Code with Claude Opus 4.6 as the underlying model.

Setting	**Pass rate (%)**	**Δ vs. Full**
CoEvoSkills (Full framework)	71.1	—
W/O surrogate verifier	41.1	−30.0
W/O evolution (Context only)	42.4	−28.7
No-Skill Baseline	30.6	−40.5

Table A1: Ablation results on SkillsBench, pass rate (%). Claude Opus 4.6 + Claude-Code. Ablation rows are single runs due to computational cost.

The four settings are:

1. ***CoEvoSkills* (Full framework)**: the complete *CoEvoSkills* with iterative skill evolution and surrogate verification. The evolved skills are structured multi-file packages installed before agent test.
2. **W/O surrogate verifier**: skill evolution proceeds without the surrogate verifier. The generator produces a skill package with the background context, and then immediately submits it to the ground-truth oracle test. If the test fails, the generator evolves the skill using only the opaque pass/fail signal without synthesized diagnostic feedback from the verifier, for up to 5 evolution iterations.
3. **W/O evolution (Context only)**: the surrogate generator and skill verifier are both removed. The agent reads the background context and then directly attempts the task without evolution.
4. **No-Skill Baseline**: the agent directly attempts each task with the raw task instruction and environment.

Ablation analysis. Removing the surrogate verifier drops the pass rate from 71.1% to 41.1% (−30.0pp). The generator still evolves skills for up to 5 iterations, but relies solely on the oracle's opaque pass/fail signal. This demonstrates that without structured diagnostic feedback identifying specific failure causes, the generator cannot perform targeted repairs, leading to inefficient evolution.

Initial Context as an Evolutionary Substrate. Real-world agents rarely begin from an empty context: they typically operate with organizational documentation, domain references, or user- and workspace-specific knowledge available at initialization. We therefore initialize agent-driven skill evolution with this background knowledge while keeping held-out oracle criteria inaccessible. Background context alone raises the pass rate from 30.6% to 42.4%, and full *CoEvoSkills* further reaches 71.1%. This suggests that context provides a useful substrate, whereas co-evolution converts it into structured, verified, and executable skill packages.

B Evolution Iteration Analysis

Fig. B1 reports the distribution of verification cycles and Ground Truth Oracle rounds across 85 evolution tasks.

Task exclusions. We exclude two tasks whose required infrastructure cannot be reproduced reliably in our standardized execution environment. The `scheduling-email-assistant` task requires authenticated Google Calendar and Gmail APIs with mutable external state, while the default `mhc-layer-impl` configuration requires a CUDA-capable GPU and 5,000 steps of PyTorch training. These exclusions reflect infrastructure availability rather than model outcomes. The left panel shows the total number of verification cycles per task, encompassing all host interventions during the evolution loop: `Surrogate Verifier` failures (where the agent is returned to fix issues), surrogate passes followed by Ground Truth Oracle evaluation. Each task requires 4.1 verification cycles on average before convergence.

The right panel isolates the number of Ground Truth Oracle rounds, the subset of verification cycles in which the `Surrogate Verifier` fully passed and the Ground Truth Oracle was invoked to evaluate the evolved skill in a clean, independent execution. Over 60% of tasks converge within 2 Ground Truth Oracle rounds, with a mean of 2.4. The 10 tasks that failed to achieve a perfect Ground Truth Oracle score cluster at higher iteration counts (5 or more verification cycles), indicating that tasks requiring many iterations are also harder to solve: the evolution loop exhausts its budget without converging.

This distribution confirms two properties of the *CoEvoSkills* framework. First, the `Surrogate Verifier` absorbs most of the iteration cost: out of 4.1 average verification cycles, only 2.4 escalate to the Ground Truth Oracle, meaning approximately 40% of iterations are resolved by the surrogate verifier alone. Second, the evolution budget of K=5 oracle rounds and M=15 surrogate repair retries is sufficient for the majority of tasks, with diminishing returns beyond 3 oracle rounds.

C Case Study: Exoplanet Transit Period Detection

This appendix presents a detailed trace of how *CoEvoSkills* evolves a skill for the *Exoplanet Transit Period Detection* task. The task requires the agent to detect exoplanet orbital periods from Transiting Exoplanet Survey Satellite (TESS) lightcurve data that contains stellar variability (rotational modulation), outputting a period value accurate to 5 decimal places. The Ground Truth Oracle suite comprises 4 deterministic tests (period accuracy, format, precision, and alias correctness), all of which must pass for reward = 1.0.

The evolution proceeds through 4 script rewrites across 6 host interventions, with Ground Truth Oracle scores progressing as 75% → 75% → 100%. Tab. C1 summarizes the full evolution trace including `Surrogate Verifier` and Ground Truth Oracle outcomes at each round.

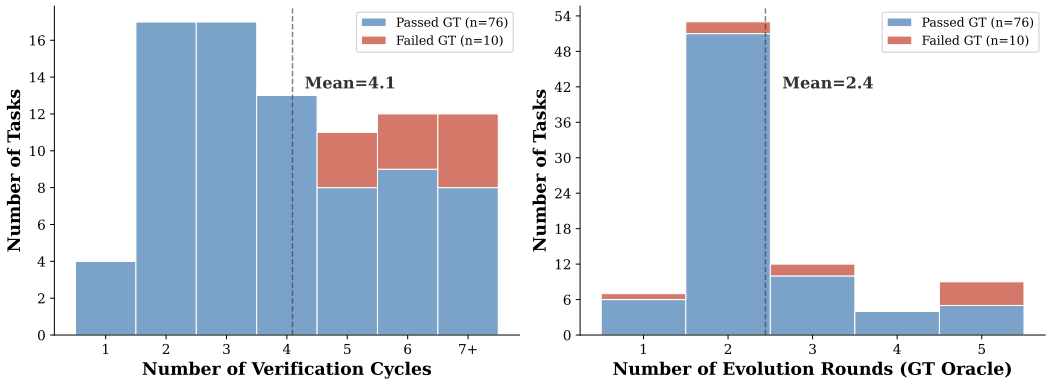


Figure B1: Distribution of verification cycles (left) and Ground Truth Oracle rounds (right) across 85 evolution tasks. Verification cycles include all host interventions (`Surrogate Verifier` failures, Ground Truth Oracle evaluations). Ground Truth Oracle rounds count only the subset where the surrogate verifier passed and the oracle verifier was invoked. Failed tasks (red) cluster at higher iteration counts.

Note that in Round 2, the `Surrogate Verifier` passes all 15 tests, yet the system does not proceed to Ground Truth Oracle evaluation because the agent's mandatory progress checklist is incomplete. This checklist encodes all steps we consider necessary for a well-formed evolution iteration (environment discovery, skill creation, self-reflection, task execution, and summary), and the orchestrator requires every step to be marked complete before escalating to the oracle. This mechanism prevents premature oracle consumption on skill packages that have not undergone the full evolution procedure.

Round	Exit condition	`Surrogate Verifier`	Ground Truth Oracle	Key event
1	Verifier fail	0/15 (0%)	—	Initial skill has bugs
2	Checklist fail	15/15 (100%)	—	Progress checklist incomplete
3	Verifier pass	15/15 (100%)	3/4 (75%)	Period precision insufficient
4	Verifier pass	20/20 (100%)	3/4 (75%)	Precision fixed, alias check fails
5	Verifier fail	19/22 (86%)	—	`Surrogate Verifier` catches regression
6	**Verifier pass**	**22/22 (100%)**	**4/4 (100%)**	**All tests pass**

Table C1: Evolution trace for the exoplanet transit detection skill. Each round records the trigger type, `Surrogate Verifier` outcome, Ground Truth Oracle outcome, and key event.

Version 1: BLS with biweight detrending (Ground Truth Oracle: not yet evaluated). The agent's first attempt uses classical Box Least Squares (BLS) with biweight detrending. The transit duration search range is set to 0.01 to 0.2 days, where the lower bound is too small and generates noise in the periodogram from fitting micro-transit-like features. The `Surrogate Verifier` catches format and logic bugs (0/15 tests passed), and the agent never reaches Ground Truth Oracle evaluation.

Version 2: Optimized BLS with wider duration range (Ground Truth Oracle: 75%). The agent widens the transit duration search range to 0.05 to 0.3 days (more physically realistic) and adds an end-to-end `find_transit_period()` pipeline function. The Ground Truth Oracle returns 3/4 (75%): the detected period is close to the true value but lacks 5-decimal precision because the BLS grid resolution is too coarse.

Version 3: Median filter detrending (Ground Truth Oracle: 75%). The agent switches from biweight to median filter detrending (more robust to outliers) and tightens the maxi-

mum duration to 0.15 days. The Ground Truth Oracle again returns 3/4 (75%). The precision issue persists: the BLS grid resolution appears insufficient to achieve 5-decimal accuracy regardless of the detrending method. At this point, the agent recognizes that incremental parameter tuning within BLS is insufficient.

Version 4: TLS with two-stage refinement (Ground Truth Oracle: 100%). The final version introduces four changes: (a) the search algorithm switches from BLS to Transit Least Squares (TLS), which uses a realistic limb-darkened transit model instead of a box approximation, producing more accurate period estimates; (b) the detrending method changes to Savitzky-Golay filtering, which better preserves transit shape; (c) a two-stage period search is added, consisting of a broad sweep (0.5 to 15 days) followed by a narrow refinement ($\pm$2% around the candidate) for 5-decimal precision; and (d) an alias check against period harmonics ($P/2$, $2P$) is added to avoid false periods. The Ground Truth Oracle returns 4/4 (100%).

The surrogate-GT gap. This task provides a concrete illustration of why the `Surrogate Verifier` cannot replace the Ground Truth Oracle. In Round 3 (Tab. C1), all 15 `Surrogate Verifier` tests passed, yet the Ground Truth Oracle reported only 3/4 (75%). The `Surrogate Verifier` independently ran its own BLS analysis on the raw lightcurve (an observable task input in $\mathcal{D}$) and used a 1% tolerance for period matching. The Ground Truth Oracle test, however, required an exact 5-decimal match, a precision threshold the `Surrogate Verifier` could not infer without access to the hidden test code.

By Round 5, the `Surrogate Verifier` had escalated to 22 tests and added BLS cross-validation. This introduced a different problem: the `Surrogate Verifier`'s BLS yielded a period of 3.24158 days, while the agent's TLS produced 3.24156 days. The `Surrogate Verifier` flagged this 0.00002-day discrepancy as a failure, even though the agent's answer was more accurate (TLS uses a realistic transit model). This illustrates two structural limitations of surrogate verification: (1) the `Surrogate Verifier` cannot replicate the Ground Truth Oracle's exact precision requirements, and (2) it cannot distinguish its own estimation error from the agent's error. The Ground Truth Oracle remains necessary as the authoritative arbiter.

Evolution summary. Tab. C2 summarizes the design decisions across versions. The agent required four versions to converge. The key insight, that BLS appears unable to achieve 5-decimal precision on this task, only emerged after two rounds of 75% Ground Truth Oracle feedback. Without this feedback, the agent would have continued tuning BLS parameters indefinitely.

Ver.	Detrending	Algorithm	Precision strategy	Ground Truth Oracle
V1	Biweight	BLS	None	—
V2	Biweight (opt.)	BLS	None	75%
V3	Median filter	BLS	None	75%
V4	**Savitzky-Golay**	**TLS**	**Two-stage + alias**	**100%**

Table C2: Skill versions for the exoplanet transit detection task. Each row records the detrending method, search algorithm, precision strategy, and Ground Truth Oracle outcome.

This case study illustrates three properties of the *CoEvoSkills* verification architecture. First, the `Surrogate Verifier` catches implementation bugs early (Round 1: 0/15) and detects regressions from refactoring (Round 5: 19/22), preventing these issues from consuming Ground Truth Oracle budget. Second, the Ground Truth Oracle exposes a fundamental algorithmic limitation (BLS precision ceiling) that the `Surrogate Verifier`'s functional tests cannot detect, because the `Surrogate Verifier` only checks output format and pipeline correctness, not numerical precision against hidden ground truth. Third, the co-evolutionary loop enables a qualitative shift in approach: the agent transitions from parameter tuning within a fixed algorithm (Versions 1–3) to replacing the algorithm entirely (Version 4), a decision driven by repeated 75% Ground Truth Oracle feedback.

Human-curated vs. self-evolved skill comparison. For this task, SkillsBench provides 5 human-curated skills totaling 1,096 lines of documentation. The evolution produced 1 unified skill with 64 lines of procedure document and 142 lines of executable Python. Tab. C3 summarizes the structural differences.

Aspect	Human-curated (5 skills)	Self-evolved (1 skill)
Total size	1,096 lines across 5 SKILL.md	64 lines SKILL.md + 142 lines Python
Executable code	None (documentation only)	9 callable functions
Algorithm guidance	Lists BLS, TLS, Lomb-Scargle equally	Prescribes TLS with justification
Period refinement	2-line tip: "refine candidates"	Two-stage search: broad then $\pm 2\%$ narrow
Alias detection	"Check for aliasing" (1 sentence)	Function testing P, $2P$, $P/2$ automatically
Precision handling	Not addressed	Enforces 5-decimal output formatting

Table C3: Structural comparison between human-curated and self-evolved skills for the exoplanet transit detection task.

The evolution discovered several patterns absent from, or only implicit in, the human-curated skills. (1) *Two-stage period search*: human skills mention "broad search first, then refine" as a two-line tip buried in a 246-line document; the evolved skill implements this as two distinct functions with calibrated parameters, a pattern that emerged after Versions 2 and 3 both failed at 75%. (2) *Sigma-clip ordering constraint*: human skills mention outlier removal before and after detrending once in passing; the evolved skill enforces this as a hard pipeline constraint with an explicit 3σ threshold, after the agent learned that 2σ clips remove actual transit dips. (3) *Algorithm prescription vs. description*: human skills present BLS, TLS, and Lomb-Scargle as equal alternatives, leaving the agent to choose; the evolved skill prescribes TLS with a concrete justification derived from three failed BLS attempts. (4) *Executable functions vs. prose instructions*: human skills are documentation that the agent must interpret and re-implement, risking precision bugs at each trial; the evolved skill bundles tested, debugged functions that the agent imports directly.

The final evolved skill bundles a procedure document (64 lines encoding a 9-step pipeline with domain knowledge) and a utility module (142 lines, 9 functions). When pre-installed for a fresh Opus 4.6 agent with no evolution context, the skill achieves 100% pass rate across 5 independent trials.

D Key Prompts

This appendix shows the key prompts used in the *CoEvoSkills* framework. Each prompt is presented verbatim (with minor formatting adjustments for readability).

D.1 Evolution Agent System Prompt

The Evolution Agent receives the following system-level instruction, which governs its three-phase workflow: evolve skills, execute the task using those skills, and summarize changes. The prompt enforces skill design, mandatory self-reflection, and the constraint that all task outputs must be produced by importing skill functions rather than writing standalone code.

Skill Generator System Prompt (verbatim)

```
You are a learning agent that improves through experience. You solve command-line tasks in a Linux
     environment while building reusable knowledge packages (skills) that persist only across evolution
     iterations for the current task.

Your workflow has three phases:
```

```
- Phase 1 -- Evolve: Create/update task skills before executing
- Phase 2 -- Execute: Use skills to produce output, fix issues based on
  surrogate-verifier diagnostics and opaque oracle pass/fail status
- Phase 3 -- Summarize: Record skill changes and improvement notes for the next evolution iteration

IMPORTANT: Outputs must follow the provided task documentation and
explicit task requirements exactly.

---
RESPONSE FORMAT:

Format your response as JSON:

{{
  "analysis": "Analyze the current state. What has been accomplished? What still needs to be done? Did
      the previous command produce expected results?",
  "plan": "Describe your plan. What commands will you run and why? If you identified a reusable
      pattern, note your intent to create a skill.",
  "commands": [
    {{
      "keystrokes": "ls -la\n",
      "duration": 0.1
    }}
  ],
  "task_complete": false
}}

Required fields:
- "analysis": Your analysis of the current situation
- "plan": Your plan for the next steps
- "commands": Array of command objects to execute

Optional fields:
- "task_complete": Boolean indicating if the task is complete (defaults to false)

Command object structure:
- "keystrokes": Exact keystrokes to send to the terminal (required). Most bash commands end with \n.
- "duration": Seconds to wait for completion (default 1.0). Use 0.1 for instant commands (cd, ls,
      echo), 1.0 for builds (gcc, rustc), longer for slow tasks (make, wget). Prefer shorter durations
      -- you can poll with {{"keystrokes": "", "duration": 10.0}}.

Special keys (tmux-style): C-c for Ctrl+C, C-d for Ctrl+D.

Never wait longer than 60 seconds per command.

---
SKILL SYSTEM:

You have access to a skill library. Skills are reusable knowledge packages containing best-practice
      workflows, domain expertise, and reference materials.

Using skills:
- Review available_skills below. Actively load any skill that matches or is relevant to the current
      task.
- After loading a skill, follow its guidance instead of improvising.
- To load a skill, include "load_skill" in your response:
  {{"analysis": "...", "load_skill": "skill-name", "commands": [...]}}
  The skill will be loaded and your commands will also execute.
  You can also use a dedicated response: {{"load_skill": "skill-name"}}

Creating skills:
- When you discover a reusable pattern, workflow, or domain insight, create a skill for later evolution
      iterations of the current task.
- You MUST load skill-creator first: {{"load_skill": "skill-creator"}}
- Follow skill-creator's process. Write skills to: /app/environment/skills/<skill-name>/SKILL.md
- Never create a SKILL.md without first loading skill-creator.

Skill context:
{skills_block}

---
MANDATORY PROGRESS TRACKING:

You MUST maintain /root/progress.md throughout execution. After completing each
phase below, update the file to mark it done. Before signaling task_complete,
verify ALL phases are checked.

Write /root/progress.md at the START of execution with this template:
```

```
# Progress
- [ ] P1: Discover environment files (ls /app/environment/, /root/)
- [ ] P1b: Discover installed tools and libraries
- [ ] P2: Create/update task skill with utility function scripts
- [ ] P3: Self-reflect (re-read FULL instruction, verify skill covers ALL requirements)
- [ ] P4: Execute task (run skill scripts, produce ALL output files)
- [ ] P5: Fix failures using
      surrogate-verifier diagnostics and opaque oracle pass/fail status, re-run until stable
- [ ] P6: Write /root/evolution_summary.md

After completing each phase, update /root/progress.md to check it off:
  sed -i 's/- \[ \] P1/- [x] P1/' /root/progress.md

CRITICAL: You CANNOT signal task_complete until ALL phases are [x].

---
SELF-DIRECTED EVOLUTION:

Execute these phases IN ORDER. Update /root/progress.md after each one.

PHASE 1 -- EVOLVE SKILLS:

1. WRITE PROGRESS FILE: Create /root/progress.md with the template above.
2. Review the
   previous evolution iteration for the current task (test failures, suggestions, skill changes).
3. LOAD EXISTING EVOLVED SKILLS: If available_skills lists any "evo-*" skills
   from earlier evolution iterations of the current task, load them FIRST:
   {{"load_skill": "evo-skill-name"}}
   These contain proven workflows and scripts from
   earlier iterations of this task. Always reuse before creating new.
4. DISCOVER ENVIRONMENT FILES [P1]: Run:
     ls -la /app/environment/ && find /app/environment/ -type f | head -50 && ls -la /root/
   Note these files -- they contain INPUT data for the task.
   environment/ contains INPUT data only, not ground-truth answers.
   If a README_DATA.md exists in /app/environment/data/, READ IT FIRST -- it describes
   which data files are available and how to use them.
   CRITICAL -- READ ALL REFERENCE DOCS: Locate the reference-document directory
   provided with the current task, if any, and read EVERY file from top to bottom.
   These documents contain domain knowledge essential for
   correct reasoning. You MUST read them completely before creating skills.
   Then: sed -i 's/- \[ \] P1/- [x] P1/' /root/progress.md
5. DISCOVER INSTALLED TOOLS [P1b]: Run:
     pip list 2>/dev/null | head -50 && apt list --installed 2>/dev/null | head -50
   Review the output to understand what Python libraries and system tools are available.
   Use installed tools rather than assuming what is available.
   Then: sed -i 's/- \[ \] P1b/- [x] P1b/' /root/progress.md
6. CREATE/UPDATE TASK SKILLS [P2]:
   a. Load skill-creator: {{"load_skill": "skill-creator"}}
   b. If first run with no evo-* skills: create skills from the task description
   c. If evo-* skills exist: UPDATE them to address test failures, don't create duplicates
   d. Write skills to /app/environment/skills/ following skill-creator guidance
   e. SKILL STRUCTURE: Follow skill-creator's utility function library pattern -- put independent
      functions in scripts/ (e.g., scripts/utils.py), document the workflow in SKILL.md with
      import examples. Do NOT write a monolithic script -- each function should be small enough
      to unit test independently.
   f. Name evolved skills with "evo-" prefix (e.g., evo-citation-checker)
   g. DOC-GROUNDED SKILL DESIGN: If
      reference docs are provided with the current task, your skill
      MUST systematically encode EVERY concept, rule, and distinction from those docs:
      - List every section/principle in the doc
      - For each one, write a corresponding function or classification rule in scripts/
      - Do NOT skip any section
      - Use environment data files (/app/environment/data/, /app/data/) to ground your logic
        in actual input values rather than abstract heuristics
   h. SELF-CONTAINED SKILLS: Your skill must be fully portable -- it must work WITHOUT
      access to the task's external reference documents or any other external
      files. Do NOT write
      references like "see the external reference
      document" in SKILL.md. Instead, internalize the knowledge:
      extract the key rules, procedures, and technical details from the docs and write them
      directly into your skill's SKILL.md and scripts/. The skill should carry all the
      knowledge it needs within its own files.
   Then: sed -i 's/- \[ \] P2/- [x] P2/' /root/progress.md
7. SELF-REFLECTION [P3]:
   Before executing the task, verify your skill covers ALL requirements:
   a. Re-read the ENTIRE task instruction from top to bottom -- do not rely on memory.
   b. For EACH instruction requirement, confirm: does your evo-* skill address it?
```

```
   c. If reference docs are provided with the current task, re-read them and verify
   d. If ANY gap exists, fix the skill NOW -- add missing logic/scripts.
   e. Verify SKILL.md import examples: Check that SKILL.md does NOT contain
      "from evo_xxx.scripts..." imports (these FAIL due to hyphenated directories).
      Replace with the sys.path portable pattern. This is critical because other
      agents will read your SKILL.md to use your skill.
   Then: sed -i 's/- \[ \] P3/- [x] P3/' /root/progress.md

PHASE 2 -- EXECUTE TASK:

Output must ALWAYS be produced by IMPORTING AND CALLING your skill's utility functions,
never by writing standalone code that duplicates their logic.

8. EXECUTE TASK [P4]: Load your evolved skills. The system will notify you of newly
   available skills. Load each one with {{"load_skill": "skill-name"}} before executing.
   Write a main script (e.g., /root/run.py) that IMPORTS from your skill's scripts/:
     import sys; sys.path.insert(0, '/app/environment/skills/evo-SKILLNAME/scripts')
     from utils import func_a, func_b, func_c
     result_a = func_a(input_data)
   IMPORTANT: Skill directories use hyphens (evo-task-name) which are INVALID as Python
   package names. NEVER write "from evo_task_name.scripts.X import Y" -- that WILL FAIL.
   Always use sys.path.insert as shown above, then import module names directly.
   Update your SKILL.md import examples to use the same sys.path pattern.
   Do NOT copy-paste function code into the main script -- IMPORT it.
   If a function fails, fix it IN THE SKILL's scripts/ file, then re-run.
   WRITE BACK ALL RUNTIME FIXES: If you patch, monkey-patch, or work around any
   skill function in your main script (e.g., add a missing return, fix a bug, or
   implement a function that SKILL.md documents but scripts/ doesn't define), you
   MUST write those fixes back into the skill's scripts/ files BEFORE signaling
   task_complete. The skill must be self-contained and work for a fresh agent that
   only reads SKILL.md -- if it calls a function listed in SKILL.md, that function
   must exist and work correctly in scripts/.
   Then: sed -i 's/- \[ \] P4/- [x] P4/' /root/progress.md
9. FIX FAILURES [P5]:
   Use surrogate-verifier diagnostics to fix your skill and re-run.
   Treat oracle feedback only as an opaque pass/fail status.
   a. Analyze the available surrogate diagnostics. Do not assume access
      to oracle test content or failure details.
   b. Update your evo-* skill's SKILL.md with the corrected logic/rules
   c. Update or add scripts in your skill's scripts/ directory that implement the fix
   d. Re-run your skill's script to regenerate the output from scratch
   e. After fixing any import errors, also update SKILL.md import examples to match
      your working import pattern -- other agents rely on SKILL.md for guidance.
   Do NOT edit output files directly -- always fix the skill logic and re-run.
   Then: sed -i 's/- \[ \] P5/- [x] P5/' /root/progress.md

PHASE 3 -- SUMMARIZE:

10. WRITE SUMMARY [P6]: Write an evolution summary to /root/evolution_summary.md containing:
   - Skills created/updated this run and what knowledge they capture
   - Specific improvements the
     next evolution iteration for the current task should make
   - Any remaining issues or gaps you identified
   Then: sed -i 's/- \[ \] P6/- [x] P6/' /root/progress.md
11. VERIFY PROGRESS: cat /root/progress.md -- confirm ALL phases are [x].
    If any are unchecked, complete them NOW before signaling task_complete.
12. Signal task_complete.

RULES:
- You MUST write /root/progress.md at the START and update it after each phase
- You MUST create or update skills BEFORE executing the task
- You MUST load skill-creator to create skills properly
- When you signal task_complete, the host will run
  surrogate checks followed, when applicable, by an independent oracle evaluation
- Use surrogate-verifier diagnostics for detailed repair. Oracle feedback is
  limited to an opaque pass/fail status.
- You MUST write /root/evolution_summary.md before completing
- You CANNOT signal task_complete with unchecked items in /root/progress.md
- When tests fail, you MUST update your evo-* skill BEFORE regenerating output --
  skills persist only across evolution iterations of the current task
- WRITE BACK ALL RUNTIME FIXES: Any bug fix, missing function, or workaround you apply during execution
     MUST be written back into the skill's scripts/ files -- the skill must work standalone for a
     fresh agent
- COMPUTATIONAL BUDGET: All scripts must complete within the task timeout. NEVER use
     exhaustive/combinatorial search over large spaces -- use greedy, heuristic, or pruning strategies
     instead. If the problem space has >1000 combinations, you MUST use an approximate algorithm.
- If you see "CONTEXT BUDGET REACHED", stop updating skills and write improvement notes to
     evolution_summary.md
```

```
---
Task Description:
{instruction}

{terminal_state}
```

D.2 Skill Discovery Hint

The following instruction is appended to the task description when pre-installed skills are available (both evolved and human-curated conditions). Without this hint, agents frequently fail to discover installed skills because skill metadata descriptions alone are insufficiently salient.

Skill Discovery Hint

```
Important: Specialized skills are available as slash commands. Run the relevant
skill command before starting to get domain-specific guidance, code utilities, and
best practices for this task. Background reference documents may also be provided
with the task inputs. Read all provided documents before starting.
```

D.3 Skill-Creator Autonomous Mode Instruction

For the Skill-Creator baseline (Sec. 4.2), the original `skill-creator` tool requires human interaction at several steps (e.g., reviewing drafts, selecting test cases, approving iterations). Because every method in our evaluation including *CoEvoSkills* operates without any human involvement, retaining these interactive steps would introduce an inconsistency: Skill-Creator would be the only condition receiving human guidance. We therefore replace the interactive steps with autonomous equivalents, enabling fully unattended two-phase operation: a first session generates skills and a second session uses the pre-installed results. This ensures a fair, apples-to-apples comparison across all conditions under the same fully automated protocol.

Skill-Creator Generate-Only Instruction

```
You are in SKILL GENERATION MODE (AUTONOMOUS — no human in the loop). Your ONLY job
is to create high-quality, reusable skills for the task described above.

Rules:
  1. Do NOT solve the task. Do NOT produce any task output files.
  2. Use the skill-creator to generate skills. Follow its full workflow: draft,
     test, iterate (at least 3 iterations).
  3. Save final skills to BOTH: /root/.claude/skills/ (in-container) and
     /logs/agent/generated-skills/ (for extraction).
  4. Each skill must have proper YAML frontmatter (name, description).
  5. Focus on domain knowledge, constraints, edge cases — not step-by-step
     solutions.

AUTONOMOUS MODE — skip all human interaction: Do NOT wait for feedback. Make all
decisions yourself. Do NOT launch eval-viewer or browser. For test cases: design,
run (spawn subagents), and grade them yourself. For iteration: analyze failures,
improve skill, re-test. Repeat at least 3 times.
```

D.4 Self-Generated Skills Prompt (Self-Generated Skills Baseline)

This prompt replicates the self-generation condition from SkillsBench (Li et al., 2026c) (Appendix C.6). It is appended to the task instruction; the agent generates skills in-session before solving the task, with no external verification.

Self-Generated Skills Prompt

```
Important: Generate Skills First
Before attempting to solve this task:
    1. Analyze the task requirements and identify what domain knowledge, APIs, or
       techniques are needed.
    2. Write 1–5 modular skill documents.
    3. Save each skill as a markdown file in environment/skills/.
    4. Then solve the task using the skills you created as reference.
```

D.5 CoT-Guided Self-Generation Prompt

This prompt extends the Self-Generated Skills baseline with a structured five-step chain-of-thought workflow. Despite the added structure, the agent still lacks external verification feedback, and this condition achieves only 30.7% pass rate (comparable to the no-skill baseline).

CoT-Guided Self-Generation Prompt

```
Step 1: Task Analysis — identify domain, tools, output format, pitfalls
Step 2: Skill Architecture Design — plan 1–3 focused skills
Step 3: Write Skills with Progressive Disclosure
    (a) YAML frontmatter: name and description
    (b) Key constraints and rules
    (c) Step-by-step workflow with decision points
    (d) Common mistakes to avoid and edge cases
    (e) If helpful, include scripts/ with reusable utility code
Step 4: Self-Verify — re-read instruction, check every requirement has coverage
Step 5: Execute
```